

How the app works.

Order ahead from your phone, verify with a one-time code, and watch your food go from kitchen to counter live, with the menu shown in 3D.

DESIGN DRAFT · AWAITING APPROVAL

Version 1.0 · 23 September 2026

Based on Campus Canteen Live PRD + design decisions agreed in chat

Platform Mobile web app (student, kitchen and admin)



One canteen, one live source of truth.

Students order from their phones and get a daily token number. The kitchen sees the order the moment it's placed and moves it along with one tap. Every change reaches every phone within a second, with no refresh.

Order ahead

Browse a 3D menu, build a cart and confirm with a 4-digit code sent to your phone. You get a token like **107**.

Watch it live

Your tracker shows your place in the queue, an estimated wait, and each step: Placed → Preparing → Ready.

Pick up

When it's ready your phone chimes and vibrates. Show the token, pay by cash or UPI at the counter, and you're done.

What changed from the original PRD

The PRD is still the base contract. These decisions change or add to it:

AREA	PRD SAID	THIS DESIGN DOES
Counter TV	A public /display screen on a TV with a token board and QR code	Removed. The canteen has no TV. The "Ready" alert on the student's own phone is the pickup signal.
Devices	Kitchen on tablets, admin on laptops	Mobile-first everywhere. Student, kitchen, stock and admin screens are all designed for a phone first.
Anti-prank	Nothing beyond a login	New: a 4-digit SMS code before every order. Each phone number can belong to one account only.
SMS sending	—	Your own Android phone as a free SMS gateway (textbee / SMS Gateway for Android). Codes print in the terminal during development.
Food visuals	A photo or a coloured tile with the item's initial	3D food (hybrid). Still renders on the menu grid; tap a dish to open a live Three.js model you can spin.

CONTENTS

What's inside.

Start with sections 02–04 for the student experience, 06–07 for the kitchen, 08–09 for the engineering, and 11 and 15 for the design decisions still waiting on you.

01	Who uses it
02	A student's order, start to finish
03	The screens (wireframes)
04	Phone verification (OTP)
05	The order lifecycle and the bill
06	The kitchen, on a phone
07	Stock, open/closed and admin
08	How "live" works
09	Staying correct under pressure
10	Security and privacy
11	Design direction and 3D food
12	Data, API and where orders are stored
13	Tech stack
14	Build plan
15	Decisions and open questions

Three roles. The server decides who's who.

Registering always creates a Student. Staff and admins are seeded or created by an admin. Every page and every API checks the role on the server; hiding a button is never the only protection.



Student
Anyone who registers · phone

- Browse the 3D menu
- Verify with an OTP and place orders
- Track orders live, cancel while still Placed
- Up to 3 active orders at once

[/menu](#)



Staff
Kitchen and counter · phone

- Run the live kitchen board
- Start, Ready, Collected, Reject
- Mark pickups by token number
- Change stock, availability, open/closed

[/kitchen](#)



Admin
Canteen manager · phone or laptop

- Everything staff can do
- Live dashboard: orders, revenue, prep time
- Menu, users, settings and all bills
- Can't remove their own admin role

[/admin](#)

Who sees what

INFORMATION	STUDENT	STAFF	ADMIN
Their own orders	Yes	—	—
Everyone's orders today	No	Yes, first name only	Yes
Email, roll number, phone of others	No	No	Yes
Live stats and who's online	No	Presence only	Yes

A student's order, start to finish.

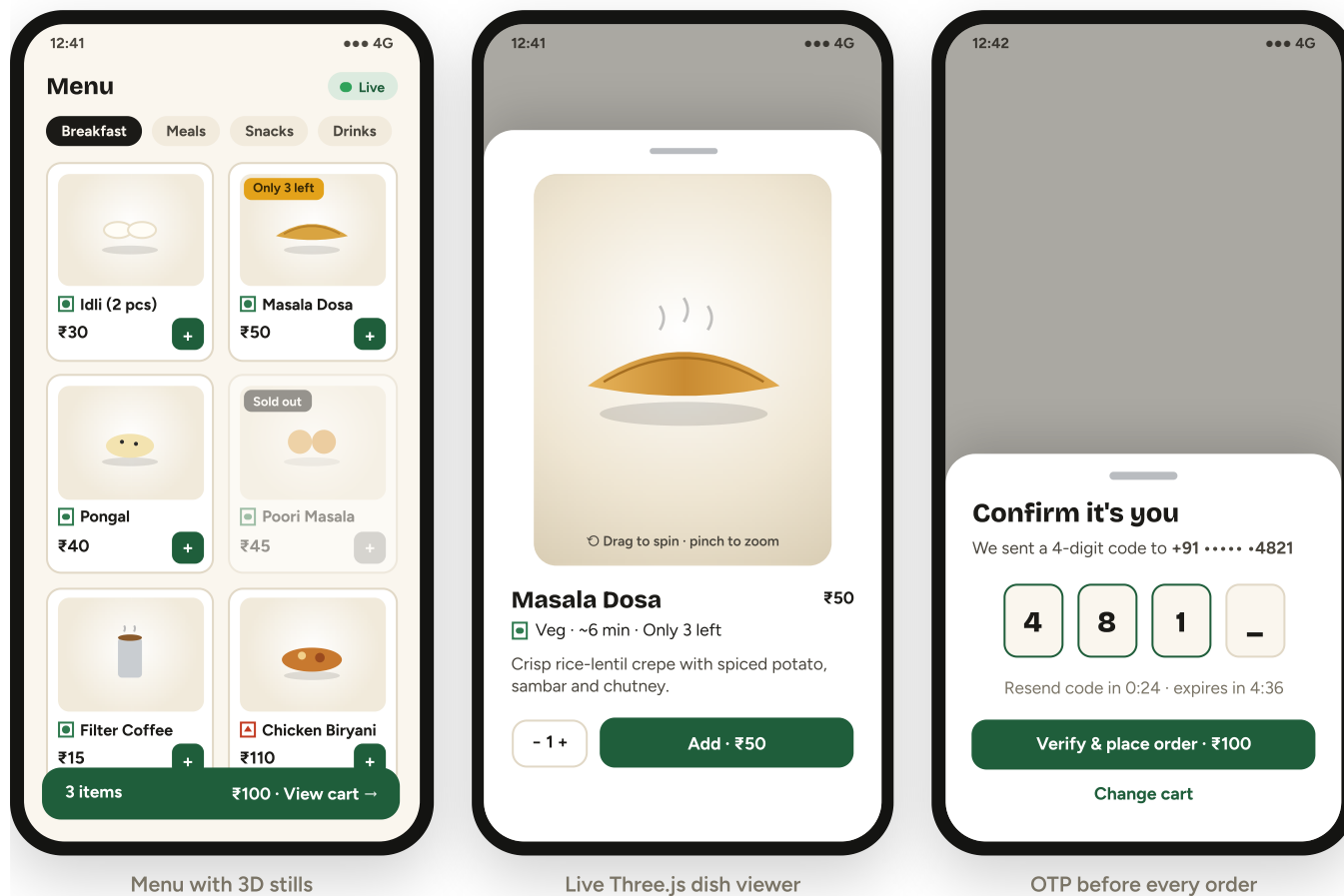
Here's what happens between opening the app and walking away with food. Each step shows what the student sees and what the system does.

- 1 **Register once.** Name, email, password, optional roll number and a **10-digit mobile number (+91)**. Each number can belong to one account only, which stops people making several accounts to send fake orders.
- 2 **Open the menu.** Dishes are grouped under sticky category tabs (Breakfast, Meals, Snacks, Beverages). Each card shows a 3D-rendered still of the dish, its price in ₹, a veg/non-veg marker, and "Only N left" or "Sold out" when relevant. All of it updates live.
- 3 **Look closer (optional).** Tap a dish and a sheet slides up with a live 3D model: drag to spin, pinch to zoom, with steam rising from hot items.
- 4 **Build the cart.** Up to 10 of any one item and 10 different items. The cart is saved on the phone, so it survives a refresh. Add a note like "less spicy".
- 5 **Place order → verify.** A 4-digit code is sent by SMS to the student's registered number. They type it in (or the phone fills it in automatically). No correct code, no order.
- 6 **Get a token.** The order is placed in a single database transaction. The student gets **token 107** and lands on the live tracker. At the same moment the order appears on every kitchen phone with a tone.
- 7 **Wait, informed.** "You are #3 in the queue · about 9 min" updates live as orders ahead move. The stepper goes Placed → Preparing.
- 8 **Ready.** The screen turns green: "Ready, show token 107 at the counter", with a chime, a vibration, and a browser notification if the tab is in the background.
- 9 **Collect and pay.** Staff type 107 into the Collect box. The student pays by cash or UPI at the counter, and the order becomes Collected. **The bill is created at that moment** and appears on the student's phone.

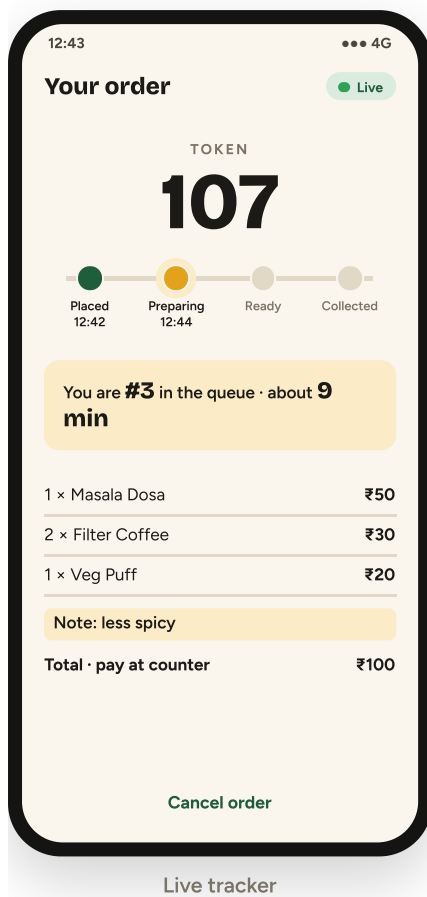
If something goes wrong: if the kitchen rejects an order, the student sees the reason ("Item ran out") and "You have not been charged", and any limited stock goes back on sale. If the canteen closes, every menu shows a banner and ordering is disabled, while orders already in progress carry on.

Wireframes of the student flow.

These show structure and behaviour, not final visuals. Final colours, 3D renders and motion follow once the 3D style is chosen (section 15).



Tracking the order to pickup.

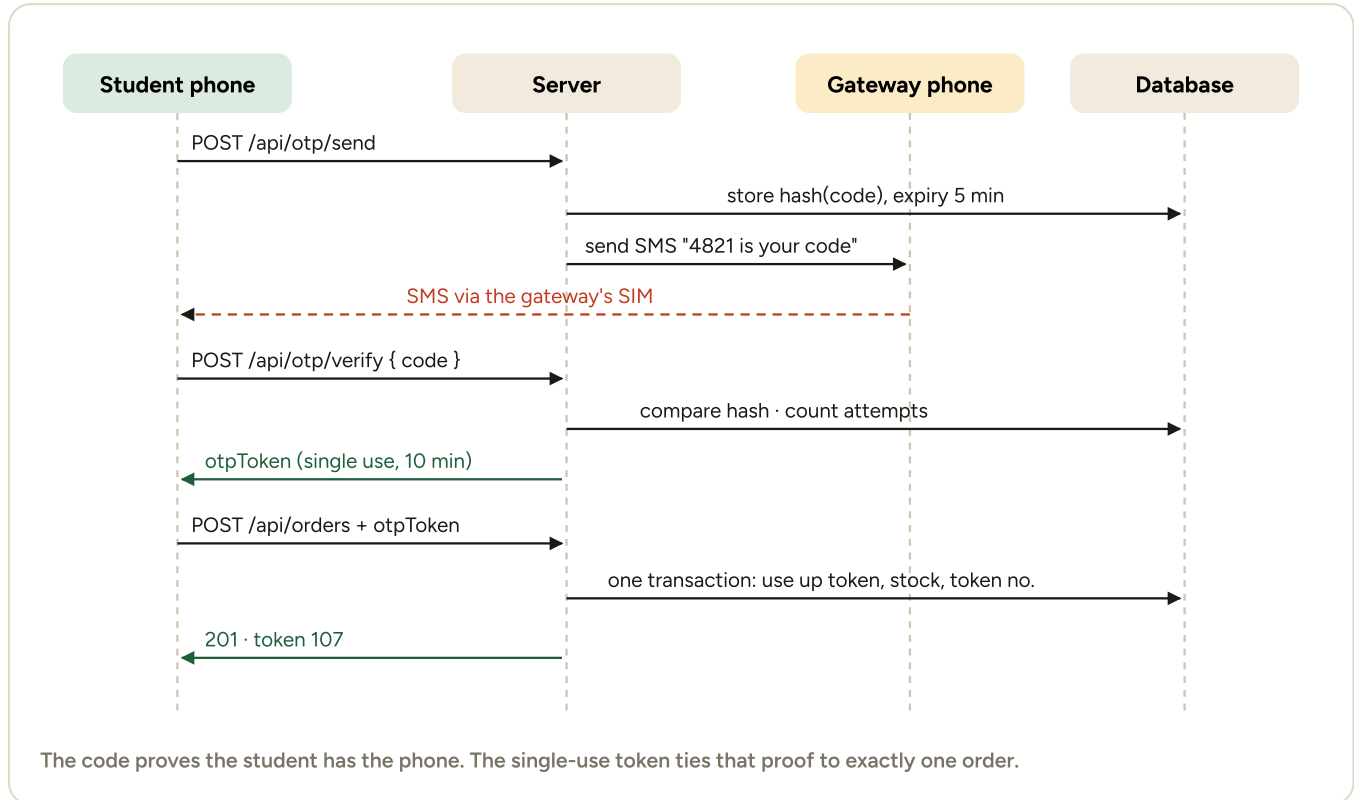


What each screen is built for

- **Menu:** built to load fast. The 3D stills are small images, so the grid is usable within 2 s on 4G.
- **Dish viewer:** 3D only loads when you ask for it. Without WebGL, or with reduced motion on, it shows the still instead.
- **OTP sheet:** one tap to request, the code fills in automatically on Android and iOS, and the resend timer is always visible.
- **Tracker:** a big token, a clear stepper with times, and the queue position. Every change is announced to screen readers.
- **Ready:** fills the whole screen so it can't be missed. Colour is never the only signal; there's always text too.

No code, no order.

So that nobody can place orders for fun, every order needs a fresh 4-digit code sent to the student's registered mobile number. The server enforces this, not just the app.



The rules

RULE	VALUE
Code length	4 digits, random
Code lifetime	5 minutes
Wrong tries allowed	5, then the code is locked
Resend cooldown	30 seconds
Send limit	5 per hour per number · 10 per hour per IP
Verified token	Single use · valid 10 minutes · this user only
Storage	Only hashes of the code and token are stored
Numbers	+91, 10 digits, one account per number

Edge cases

- **Double-tap or network retry:** the PRD's Idempotency-Key is checked *before* the token, so a retry gets back the same order instead of a "code already used" error.
- **Order fails after verifying** (e.g. out of stock): the token is only used up if the order transaction commits, so the student can fix the cart and try again without a new code.
- **SMS late at rush hour:** the resend button unlocks after 30 s, and the timer shows how long the code is valid.
- **Gateway phone offline:** the send fails with a clear message ("Couldn't send the code, try again or ask the counter") and staff get an alert.

Sending SMS for free: your own Android phone

An old Android phone with a SIM runs a free, open-source gateway app (**textbee** or **SMS Gateway for Android**). The server calls the app's API, and the phone sends the SMS from its own SIM. There are no per-message fees and no DLT registration.

Good for

- College projects and a single canteen
- Zero cost on a normal prepaid plan
- Setup takes minutes

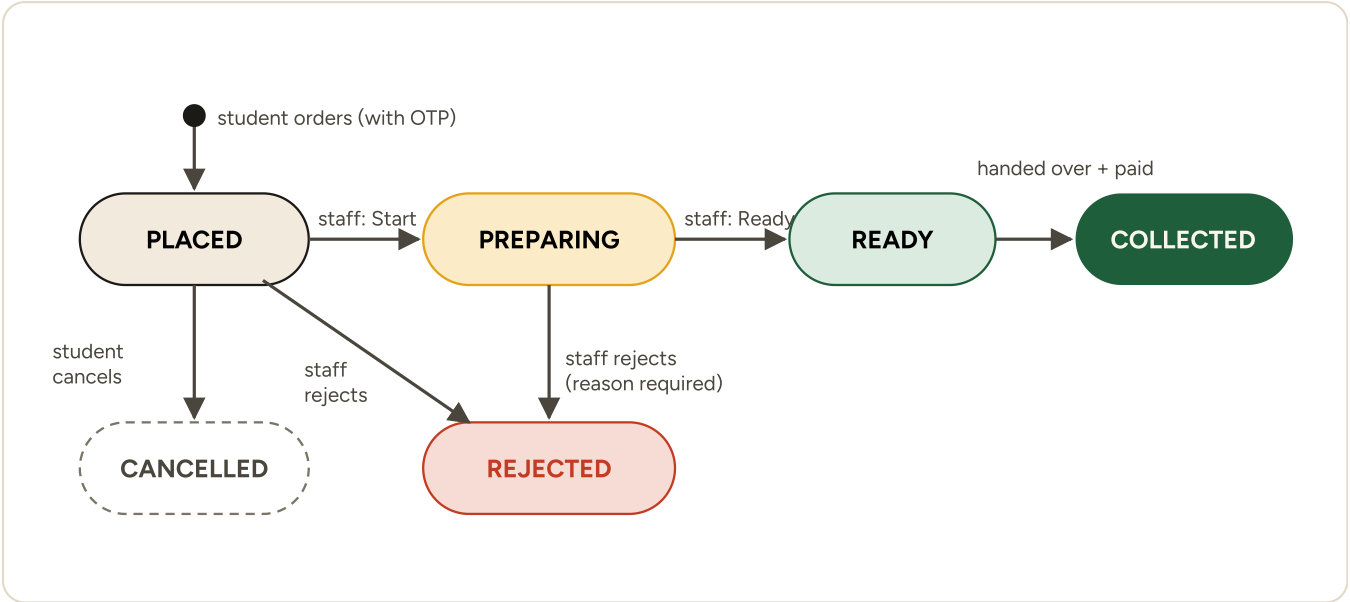
Limits to know

- Most Indian prepaid plans include **~100 free SMS a day**, so with a code per order that's about 100 orders a day
- The phone must stay on and online
- Codes come from a personal number

Built to be swapped. The SMS sender sits behind a small interface (`src/lib/sms/`) with three versions: `console` (prints the code in the terminal during development), `android-gateway`, and a paid provider (MSG91 / Twilio) added later in a single file. In development and tests a fixed `OTP_DEV_CODE` lets the simulator and tests get through; the server refuses to start in production if it's set.

Six statuses. Only these moves are allowed.

The server rejects any other move with `INVALID_TRANSITION`. Collected, Cancelled and Rejected are final.



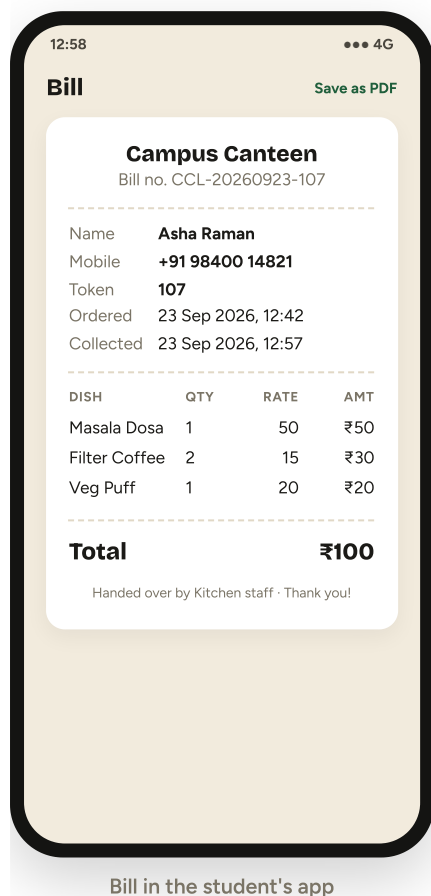
FROM → TO	WHO	WHAT ELSE HAPPENS
Placed → Preparing	Staff, Admin	Sets preparingAt. The tracker and queue positions update live.
Placed → Cancelled	The student who owns it	Limited stock is restored. The order leaves the kitchen board.
Placed or Preparing → Rejected	Staff, Admin (reason 3–140 chars)	Stock restored. The student sees the reason and "You have not been charged".
Preparing → Ready	Staff, Admin	Sets readyAt. The student's phone chimes, vibrates and notifies.
Ready → Collected	Staff, Admin	Sets collectedAt. Means it was paid for and handed over. Creates the bill in the same transaction.

Queue position and wait

The queue is today's Placed and Preparing orders, oldest first. **Position** = place in that list; **estimated wait** = position × minutes per order (default 3, set by the admin). Every time the queue changes, each waiting student gets their new position, live.

The bill: made the moment food is handed over.

When staff mark an order **Collected**, the server creates its bill in the same database transaction as the status change. The bill is stored with the order and **never changes afterwards**. Cancelled and rejected orders get no bill.



Bill in the student's app

What's on every bill

FIELD	EXAMPLE
Bill number	CCL-20260923-107 (date + token, unique)
Student name	Full name, as on the account
Mobile number	The verified number
Dishes	Name, quantity, rate, amount for each
Total	In ₹. No GST line
Times	Ordered, ready, collected (Asia/Kolkata)
Handed over by	The staff member who tapped Collected

Copied, not linked. Name, phone, dish names and prices are **copied into the bill** when it's made. If a student changes their number or the admin changes a price next week, old bills stay exactly as they were.

Who can see a bill

Student

"View bill" appears on the tracker the moment the order is Collected (live, no refresh) and stays in My orders. They can print it or save it as PDF.

Kitchen staff

Can open the bill of any order they handled today, with the phone number masked (.....•4821).

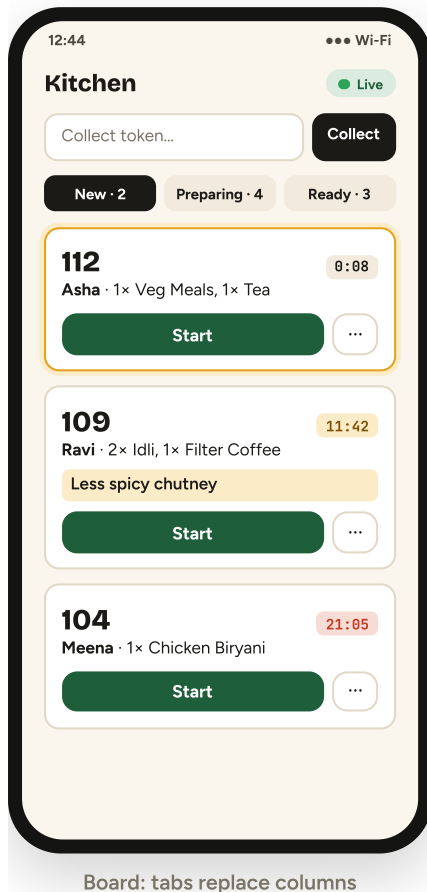
Admin

A new `/admin/bills` page lists every bill, searchable by date, bill number, name or phone, with full details.

No payment method is recorded (cash or UPI stays outside the app, as in the PRD), and there's no GST. Both can be added later without changing old bills.

The kitchen board, on a phone.

The PRD designed the board as three side-by-side columns for a tablet. On a phone the columns become three tabs with live counts, and the Collect box stays pinned at the top because it's used most at rush hour.



How it behaves

- **New orders** flash for 3 seconds and play a tone. The tab title shows "(2) New · Kitchen". Sound starts only after a tap on "Enable sound", because browsers block autoplay.
- **One big button per card:** Start → Ready → Collected. Reject is under the ... menu, with quick reasons ("Item ran out", "Kitchen closing") or a custom one.
- **Timers** tick every second: neutral under 10 min, amber from 10 to 20, red after 20. Ready cards older than 15 minutes get an "Uncollected" badge.
- **Optimistic:** the card moves the instant you tap. If another staff member got there first, it moves back and a toast says "Already updated by another staff member".
- **Collect box:** type 107 and press Enter to mark that Ready order Collected. It shows an error if 107 isn't Ready.
- **Swipe** between the New, Preparing and Ready tabs. The count on each tab updates live, so staff can see work waiting in other tabs.
- **Done today** drawer: the last 20 finished orders.
- If the connection drops for more than 3 s, a banner says "Reconnecting, board may be out of date".

Privacy: the kitchen sees only a student's **first name**, never their email, roll number or phone.

Running the canteen from a phone.

Stock and status • /kitchen/stock

- Canteen **Open / Closed** switch with an optional message ("Back at 2 PM"). When closed, every menu shows the banner and ordering is disabled.
- Every item has an availability switch and a stock number (blank means unlimited).
- Changes save automatically after 500 ms and reach every open menu live.
- Items with 5 or fewer left are highlighted.

Live dashboard • /admin

- Stat cards: orders today, revenue today (collected orders), active orders, average prep time, cancelled plus rejected.
- Orders per hour (bar chart) and top 5 items today.
- Who's online: kitchen phones and students.
- Updates at most once per second.

Menu management • /admin/menu

- Create, edit and reorder categories and items (name, description, price, veg, prep minutes, stock, availability).
- Each item is linked to its **3D model** and still image.
- Deleting archives an item: hidden from the menu, kept for order history.

Users and settings • /admin/users , /admin/settings

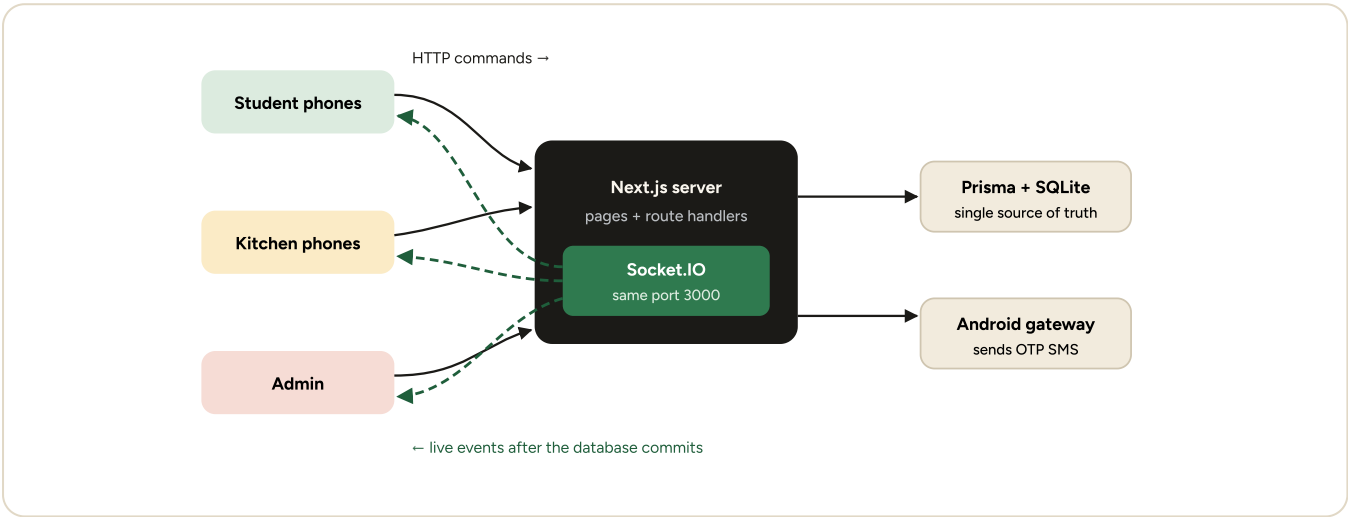
- Search users by name, email, roll number or phone. Create staff accounts and change roles.
- Settings: canteen name, minutes per order, max active orders per student, open/closed.
- **New:** the SMS gateway's status (online or offline, codes sent today against the daily limit).

Mobile-first layout rules

- Everything is designed from **360 px** wide upwards. On larger screens it gets more room, not a different app.
- Main actions sit in the bottom third of the screen, within thumb reach. Tap targets are at least 44 px.
- Tables on admin pages become stacked cards on a phone. The chart scrolls sideways inside its own box.
- Dark mode follows the system setting. Every page has a Live / Reconnecting / Offline badge in the header.

Commands go in over HTTP. Events come out over WebSocket.

Every change (placing an order, tapping Ready, switching an item off) is an HTTP request the server checks and saves in a transaction. Only *after* it's saved does the server push an event to the right people over Socket.IO. Phones never write through the socket.



Rooms: who hears what

ROOM	MEMBERS
public	Every connection: menu and open/closed
user:{id}	All tabs of one student: their orders and queue position
kitchen	Staff and admin: new and updated orders
admin	Admins: live stats and presence

The server puts each connection in its rooms based on the login cookie. Clients can't choose. The PRD's `display` room is removed along with the TV.

Events

EVENT	SENT WHEN
order:created	An order is placed
order:updated	Any status change
queue:update	The queue gains or loses an order
menu:item-updated	Stock, availability, price or name changes
menu:changed	Items or categories added, archived or reordered
canteen:status	Open/closed or settings change
stats:update	Any order change, at most once a second
presence:update	Someone connects or disconnects

Snapshot + events. Each screen first loads a **snapshot** over HTTP, then applies events on top of it. Whenever the connection comes back (after a dropped signal or a server restart), the screen reloads its snapshot, so anything missed while offline shows up. No screen ever polls on a timer.

Correct at the lunch rush, not only in the demo.

The hard part isn't the happy path. It's two people tapping at the same moment, and phones losing signal in a crowded canteen.

WHAT HAPPENS	HOW THE APP STAYS CORRECT
Two students order the last Samosa at the same moment	Stock is reduced with a conditional update ("only if stock $\geq$ quantity") inside the order transaction. Exactly one order succeeds; the other gets OUT_OF_STOCK . Stock never goes below 0.
Two staff tap Ready on the same card	Every order has a version number. The second tap carries an out-of-date version, gets VERSION_CONFLICT , and its card moves back to the server's state.
Student double-taps Place order, or the network retries	An Idempotency-Key is created when the cart opens, and the database allows each key only once. The retry returns the same order.
The same OTP token is used for two orders	Using up the token is also a conditional update inside the order transaction. Only one order can use it; the other gets OTP_REQUIRED .
Two orders need a token number at once	A daily counter is incremented inside the transaction, and the database won't accept a duplicate (date, token). Tokens restart at 101 each day.
Events arrive out of order	The phone ignores any order event whose version isn't newer than what it already has.
A price is changed in the app code	Ignored. Totals are always calculated from database prices.
Many writes hit SQLite at once	WAL mode and a 5-second busy timeout are set at startup.

When the connection drops

Reconnecting

The header badge turns amber. Socket.IO retries automatically with backoff.

Back online

Every screen reloads its snapshot, so changes made while it was offline appear without a manual refresh.

Server restart

All phones reconnect and resync by themselves. On shutdown the server closes sockets and the database cleanly.

Performance targets

- 95% of events reach phones within **1 second** on the local network.
- **200** connected phones on a normal laptop.
- The student menu is usable within **2 seconds** on a mid-range phone over 4G. The 3D stills are small images, and the Three.js viewer only loads when a dish is opened.

Trust the server, never the phone.

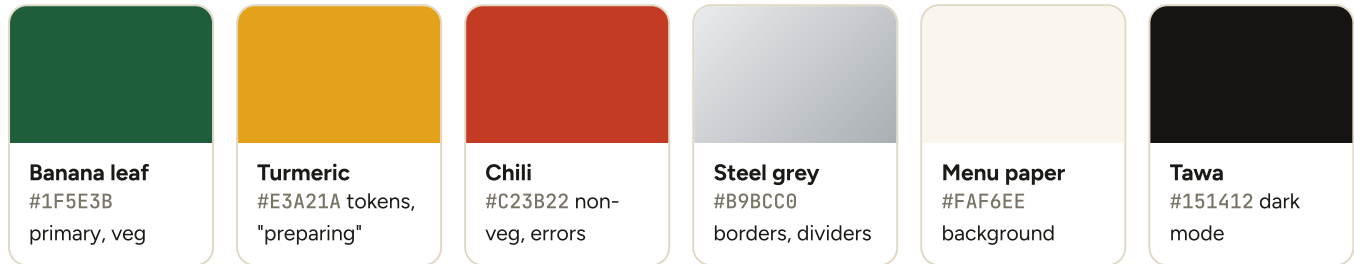
- **Sessions:** a signed JWT in an httpOnly, SameSite=Lax cookie, valid 7 days (Secure in production). The server won't start without a strong secret.
- **Passwords** are hashed with bcrypt. Login errors never reveal whether an email exists.
- **Role checks** happen in every route handler and server layout, never only in the UI.
- **Server-side checks:** prices, roles, stock, status and OTP are all checked again on the server.
- Write requests accept only JSON, which blocks cross-site form posts.
- **Rate limits:** 5 orders a minute per user, 10 logins a minute per IP, plus the OTP send limits (section 04). Too many requests get `RATE_LIMITED`.
- **OTP:** codes and tokens are stored only as hashes. Codes are never logged in production.
- **Privacy:** the kitchen sees first names only. Only admins see email, roll number and phone. Phone numbers are masked on screen (••••• •4821).
- **Redirects:** the `?next=` redirect accepts only relative paths.
- Error responses never include stack traces.

Accessibility

- WCAG 2.1 AA contrast, visible focus rings, and a kitchen board that works with a keyboard.
- Status is never shown by colour alone; every state has text or an icon. Changes are announced with an aria-live region.
- Reduced motion turns off pulses, slide animations and 3D auto-rotation, and shows 3D stills instead of live models.
- The OTP inputs work as one field for screen readers and support the phone's automatic code fill.

Banana leaf, turmeric, chili.

The look draws on a real South Indian college canteen: banana-leaf green, turmeric yellow, chili red and a warm off-white like a paper menu. The food is the hero: every dish is shown on its own, with no plates or props. The aim is a friendly, appetising app that feels local, not a generic food-delivery template.



Typography

Bricolage Grotesque

Headings and big token numbers: warm, a little quirky, very legible.

Figtree

Body text and UI: clear at small sizes on phones.

107 • 11:42

Tabular numbers for timers and tokens, so digits don't jump.

Motion

- Cards slide between kitchen tabs. New orders arrive with a soft turmeric flash.
- The tracker stepper fills like a progress line. Ready bursts into full-screen green.
- Spring physics on sheets and drags so they feel physical.
- Everything respects reduced motion.

3D food: the hybrid approach (agreed)

1 • One model per dish

Each of the 16 dishes gets a 3D model of **the food alone**, **with no plate**, floating over a soft shadow with matching lighting. Drinks keep their cup.

2 • Stills for the menu

A render script photographs each model once into a small WebP (~30 KB). The menu grid uses these, so it stays fast on 4G.

3 • Live viewer on tap

Tapping a dish loads the Three.js viewer (react-three-fiber + drei): drag to spin, pinch to zoom, steam on hot items, soft shadow.

Still to decide: the style of the models. (a) **Stylised, built in code:** clay or toy-like, tiny files, perfectly consistent, fully controllable. (b) **Realistic, AI-generated from food photos** (e.g. image-to-3D models): closer to real food, textured, ~1–3 MB each, and the quality varies so they need cleanup. Side-by-side previews will be shown before choosing.

What's added to the PRD's contract.

All PRD models, endpoints, event names and error codes stay exactly as written, except the display pieces, which are removed. These are the additions for phone verification and 3D.

Data model changes

MODEL	CHANGE
User	+ phone <code>String? @unique</code> : required for students at registration (+91, 10 digits); optional for staff and admin.
Bill (new)	<code>id, billNumber @unique, orderId @unique, tokenNumber, studentName, studentPhone, totalPaise, orderedAt, readyAt, collectedAt, collectedByName, createdAt</code> : copied values, never updated
BillLine (new)	<code>id, billId, name, quantity, unitPricePaise, lineTotalPaise</code>
OtpChallenge (new)	<code>id, userId, phone, codeHash, expiresAt, attempts, verifiedAt?, tokenHash?, tokenExpiresAt?, usedAt?, orderId?, createdAt</code>
MenuItem	+ modelUrl <code>String?</code> (3D model) · the existing <code>imageUrl</code> holds the rendered still.

New endpoints

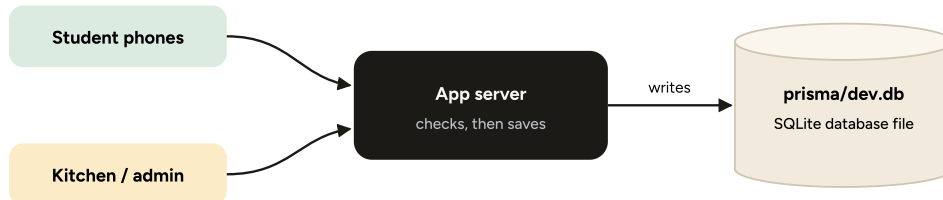
METHOD	PATH	WHO	PURPOSE
POST	<code>/api/otp/send</code>	Student	Send a code to the account's phone. Returns expiry and resend time.
POST	<code>/api/otp/verify</code>	Student	Body <code>{ code }</code> . Returns <code>{ otpToken, expiresAt }</code> .
POST	<code>/api/orders</code>	Student	Changed: body also needs <code>otpToken</code> . The Idempotency-Key header is still required.
GET	<code>/api/orders/[id]/bill</code>	Owner, Staff (masked), Admin	The bill of a Collected order; 404 before that.
GET	<code>/api/admin/bills</code>	Admin	Search bills by date, bill number, name or phone (paginated).
GET	<code>/api/admin/sms-status</code>	Admin	Gateway online or offline, and codes sent today.

New error codes

CODE	HTTP	WHEN
<code>OTP_REQUIRED</code>	403	Order sent without a valid, unused, unexpired OTP token
<code>OTP_INVALID</code>	400	Wrong or expired code; details include tries left
<code>OTP_LOCKED</code>	429	5 wrong tries: request a new code
<code>PHONE_TAKEN</code>	409	That number already belongs to another account
<code>SMS_SEND_FAILED</code>	503	The gateway couldn't send the SMS

Where the order records are stored.

Every order is stored in **one place: a database file on the computer or server running the app**. Phones only show copies. If a phone is lost or the app is closed, no order data is lost.



Phones send requests. Only the server writes to the database file.

What's recorded for each order

TABLE	WHAT IT HOLDS
Order	Token number, business date, status, total (in paise), note, reject reason, which student, and the time of every step: placed, preparing, ready, collected, closed.
OrderItem	Each line of the order: item, quantity, and a copy of the name and price at the time of ordering , so later menu edits never change old orders.
DailyCounter	The last token number used each day (starts at 101).
Bill , BillLine	The bill of every collected order: bill number, student name and mobile, dishes with rate and amount, total, and the times. Never edited.
OtpChallenge	Each verification: the phone number, a hash of the code (never the code itself), tries, and which order it was used for.
User , MenuItem , Settings	Accounts, the menu and stock, and canteen settings.

How long it's kept

- **Forever.** Nothing is deleted automatically.
- Cancelled and rejected orders are kept, with their reason.
- Students see 30 days in My orders; older orders stay in the database.

What stays on phones

- Only the **cart**, until the order is placed.
- A temporary copy of the screen that disappears when the app closes.
- The gateway phone keeps the sent code SMS.

Keep the file safe

- Back up by copying `dev.db` while the app is stopped.
- `npm run db:reset` **wipes all orders.**
- Free cloud hosts wipe the disk on redeploy, so use a persistent disk or switch to **PostgreSQL** (a one-line change).

One Next.js project, one Node process.

LAYER	CHOICE
Framework	Next.js (latest stable), App Router, TypeScript strict
Real-time	Socket.IO 4 on the same HTTP server and port, via a custom <code>server.ts</code> run with tsx
Database	SQLite with Prisma 6 (can switch to PostgreSQL by changing the provider)
Auth	JWT (jose) in an httpOnly cookie · bcryptjs
OTP / SMS NEW	Swappable SMS module: console (dev) · Android gateway (textbee / SMS Gateway for Android) · paid provider later
3D NEW	Three.js via react-three-fiber + drei, loaded only when needed · glTF models compressed with Draco/Meshopt · render script for WebP stills
Validation	Zod, shared by the API and the forms
Client data	TanStack Query v5 (socket events update its cache) · Zustand cart saved on the phone
UI	Tailwind CSS, shadcn/ui, lucide icons, sonner toasts, Motion for springs
Charts, dates	Recharts · date-fns + date-fns-tz (Asia/Kolkata)
Tests	Vitest (unit + integration) · Playwright (multi-browser end to end)

Hosting: needs a server that keeps a Node process running, such as Render, Railway, a VPS or your laptop on the campus Wi-Fi. Vercel can't hold WebSocket connections open, so it isn't a target. For the demo, phones on the same Wi-Fi open `http://<laptop-IP>:3000`.

Project location: keep the code **outside OneDrive** (e.g. `C:\Users\sreev\projects\campus-canteen-live`). OneDrive tries to sync `node_modules` and the SQLite file, which is slow and can lock files.

Nine milestones, each one checked.

After every milestone: typecheck, lint and tests must pass, then a commit.

M1 Scaffold and live socket

Next.js + Tailwind + shadcn, Prisma schema (with phone and OTP tables), seed data, `server.ts` with Socket.IO, header connection badge, design tokens and fonts.

Done when: setup + dev work on a fresh clone; the badge goes green, amber when the server stops, and green again after a restart.

M2 Auth and roles + PHONE

Register (with a unique phone number), login, logout, me, JWT cookie, role layouts, socket auth and rooms, role redirects.

Done when: each seeded account lands on its home page; students are kept out of /kitchen; the wrong role gets 403.

M3 Menu, cart and menu admin

GET /api/menu, the mobile menu with stills (placeholder renders at first), saved cart, admin menu CRUD, /kitchen/stock.

Done when: a student can build a cart; admin edits show after a refresh.

M4 OTP and ordering + OTP

SMS module (console + Android gateway), send and verify with limits, placeOrder with the token used up in the transaction, tracker, My orders, cancel.

Done when: no order without a valid code; token 101; stock drops; a double submit makes one order; cancel restores stock.

M5 Real-time core

Emitters, mobile kitchen board with transitions and version checks, Collect box, live order tracking, **bill generated on Collected** with the bill view and admin bills list.

Done when: with a student phone and a kitchen phone side by side, every change shows on both within 1 s, and collecting an order shows its bill on the student's phone.

M6 Live extras

Live menu updates, open/closed, queue position and wait, sounds and vibration, notifications, admin dashboard with stats and presence.

Done when: every Live item works in two browsers side by side.

M7 3D food NEW

16 dish models in the chosen style, render script for the stills, live Three.js viewer sheet, fallbacks for low-power devices, no WebGL and reduced motion.

Done when: the menu is usable in < 2 s on throttled 4G and the viewer runs smoothly on a mid-range phone.

M8 Resilience and hardening

Refetch on reconnect, out-of-order protection, optimistic kitchen actions with rollback, rate limits (including OTP), error shape everywhere, graceful shutdown, SQLite settings.

Done when: stopping the server for 10 s while staff change orders ends with every phone correct after the restart.

M9 Tests, simulator and docs

All PRD tests (minus display) plus OTP tests, simulator using the dev code, README with LAN demo and gateway phone setup, DECISIONS.md.

Done when: every acceptance check passes and `npm run simulate -- --kitchen` runs a live demo.

What's agreed, and what's still open.

Agreed

DECISION	CHOICE
Platform	Mobile web app, mobile-first for every role
Counter TV	Removed. The pickup signal is on the student's phone
3D scope	Food only
3D approach	Hybrid: stills on the menu, live Three.js viewer on tap
3D presentation	Food only, no plates: each dish floats over a soft shadow
Anti-prank	4-digit SMS code before every order; one account per phone number
SMS sending	Free: own Android phone as the gateway; console in development
Bill	Created on Collected: bill no., name, mobile, dishes, rates, total, times. No payment method, no GST
Engineering base	The PRD's contract, milestones and tests, adjusted as above

Still open

QUESTION	OPTIONS · RECOMMENDATION
Style of the 3D food models	Stylised and built in code vs realistic AI-generated from photos. Previews first, then choose.
Visual direction	"Banana leaf, turmeric, chili" (section 11), proposed and waiting for your approval
Gateway app	textbee vs SMS Gateway for Android. Recommend SMS Gateway for Android in local mode (no account needed)
Project folder	Recommend <code>C:\Users\sreev\projects\campus-canteen-live</code> (outside OneDrive)

Removed from the PRD

`/display` page · `GET /api/display` · `display` room · `display:update` event · `DisplaySnapshot` type · QR code · display counts in presence · display socket tests.

Next step: once the open questions are settled and you approve this document, the plan becomes the implementation spec and building starts at M1.